Museum Heist

Grid-based stealth game integrating Greedy, Backtracking, Linked List, BST, C++, Python, Pygame, and JSON

Computer Science I - 2026-I

Navarro De La Rosa - Garrido Medina - Urrego Lopez

Introduction

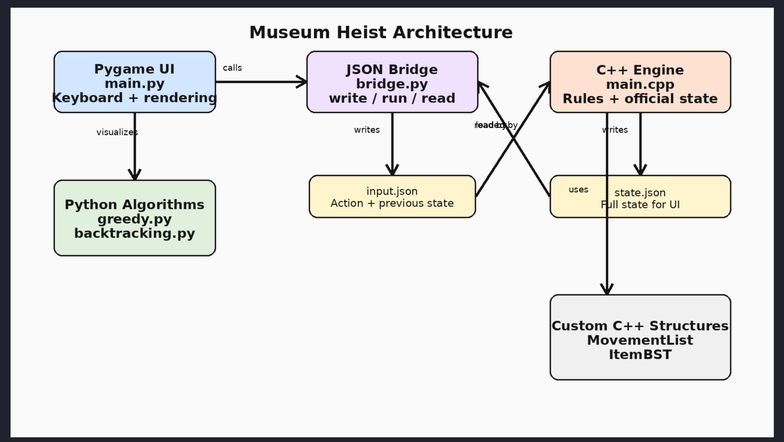
Museum Heist is a playable stealth game built for Computer Science I. The player moves through a museum grid, collects items, avoids surveillance, and reaches the exit before losing by alarm, movement limit, or time limit.

The project turns course topics into mechanics: greedy recommends an item, backtracking shows a route, C++ structures store state, and Pygame renders the game.

Goal

Collect valuable items, avoid guards, cameras, and vision zones, then escape through the exit with the best possible final metrics.

Architecture



- Pygame UI writes input.json.
- C++ engine processes official state.
- state.json returns data to UI and algorithms.

Algorithms

Greedy item selection

Chooses one remaining item using immediate local criteria: high value, proximity to guard, and danger status. It is locally optimal, not globally guaranteed.

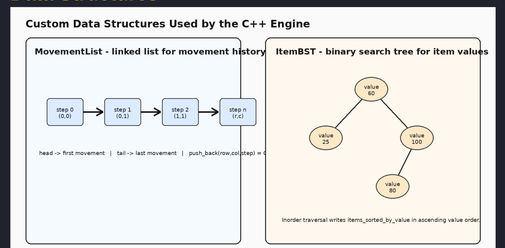
Backtracking escape route

Uses recursive choose-explore-unchoose logic to find a safe route from the player to the exit while avoiding walls, cameras, guards, and vision zones.

Complexities

Greedy: O(i(g+v)). Backtracking: exponential worst case, bounded in practice by small maps and blocked cells.

Data Structures



- MovementList: appends valid player positions and writes movement_history.
- ItemBST: orders remaining items by value and writes sorted output.

Implemented Results

Three difficulty levels: Easy, Normal, Hard.

Keyboard movement with WASD and arrows.

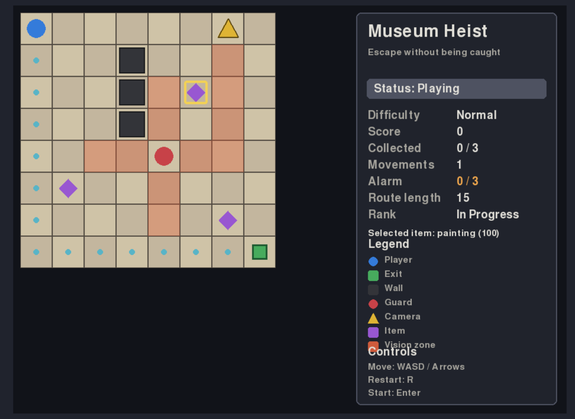
Camera and guard vision zones with wall blocking.

Item collection, disappearance, score, and final ranking.

Loss conditions: vision, alarm, movement limit, and time limit.

Final screens, tutorial, restart, and main-menu return.

Gameplay UI



Game Rules

Win: reach the exit safely.

Lose: enter vision zone, alarm limit, movement limit, or time limit.

Alarm increases after wall collisions, guard proximity, or risky item collection.

R restarts the current difficulty. M returns to the main menu.

Conclusions

Museum Heist demonstrates that algorithms and data structures can drive a playable system rather than remain isolated exercises.

The final architecture separates responsibilities clearly: C++ owns rules and structures, Python owns interaction and visualization, and JSON connects both layers.

Future work may include external map files, moving guard patrols, and optimized route search.